

Java Bean Validation - Anotaciones útiles para la validación

En **Java Bean Validation** (parte de la especificación **Jakarta EE** o **Java EE**) hay varias anotaciones que se pueden usar para validar diferentes tipos de datos en tus entidades, además de '@NotNull'. Estas anotaciones permiten asegurarse de que los datos que ingresan a tu aplicación sean válidos, evitando inconsistencias o errores antes de que los datos lleguen a la base de datos.

Aquí te dejo algunas de las anotaciones más comunes y útiles que puedes usar en tus entidades.

1. '@Size'

- Se utiliza para restringir el tamaño de una colección, mapa, array, o la longitud de una cadena de texto.

```
@Size(min = 2, max = 100,  
      message = "El título debe tener entre 2 y 100 caracteres")  
private String titulo;
```

- '**min**': Mínimo número de caracteres o elementos.
- '**max**': Máximo número de caracteres o elementos.
- '**message**': Mensaje de error personalizado.

2. '@Min' y '@Max'

- Se usan para especificar un valor mínimo o máximo para un campo numérico.

```
@Min(value = 1900,  
      message = "El año de publicación no puede ser menor a 1900")  
@Max(value = 2024,  
      message = "El año de publicación no puede ser mayor al año actual")  
private int añoPublicacion;
```

- '@Min': Define el valor mínimo permitido.
- '@Max': Define el valor máximo permitido.

3. '@Pattern'

- Se utiliza para validar una cadena de texto basada en una expresión regular (regex).

```
@Pattern(regex = "[a-zA-Z0-9]+@[a-zA-Z0-9]+\\. [a-zA-Z]{2,6}$",  
        message = "El email debe tener un formato válido")  
private String email;
```

- '**regex**': Expresión regular que define el formato válido.
- '**message**': Mensaje de error en caso de no coincidir con el patrón.

4. '@Email'

- Valida que el campo tiene un formato de correo electrónico válido. Es una validación específica para emails.

```
@Email(message = "El email debe ser válido")  
private String email;
```

5. '@Future' y '@Past'

- '@Future': Verifica que una fecha debe estar en el futuro.
- '@Past': Verifica que una fecha debe estar en el pasado.

```
@Future(message = "La fecha límite de devolución debe ser una fecha futura")
private LocalDate fechaLimiteDevolucion;

@Past(message = "La fecha de préstamo debe ser una fecha pasada")
private LocalDate fechaPrestamo;
```

6. '@Digits'

- Se usa para limitar la cantidad de dígitos enteros y decimales en un número.

```
@Digits(integer = 4, fraction = 0,
        message = "El año debe ser un número de 4 dígitos sin decimales")
private int añoPublicacion;
```

- 'integer': Número máximo de dígitos enteros permitidos.
- 'fraction': Número máximo de dígitos decimales permitidos.

7. '@Positive', '@PositiveOrZero', '@Negative', '@NegativeOrZero'

- Valida que el valor de un campo numérico es positivo, negativo, o cero.

```
@Positive(message = "El número de copias debe ser un valor positivo")
private int numeroCopias;

@PositiveOrZero(message = "El número de páginas no puede ser negativo")
private int numeroPaginas;
```

8. '@AssertTrue' y '@AssertFalse'

- Verifica que un valor booleano es verdadero o falso.

```
@AssertTrue(message = "El usuario debe aceptar los términos y condiciones")
private boolean aceptaTerminos;

@AssertFalse(message = "El campo no debe ser verdadero")
private boolean esActivo;
```

9. '@Length' (de Hibernate Validator)

- Similar a '@Size', pero específica para validar cadenas de texto. Es parte de **Hibernate Validator**.

```
@Length(min = 3, max = 100,
        message = "El nombre debe tener entre 3 y 100 caracteres")
private String nombre;
```

10. '@CreditCardNumber' (de Hibernate Validator)

- Valida que el número de una tarjeta de crédito tenga un formato válido (usando el algoritmo de Luhn).

```
@CreditCardNumber(message = "Número de tarjeta de crédito no válido")
private String tarjetaCredito;
```

11. '@CNPJ', '@CPF' (de Hibernate Validator, específico para Brasil)

- Valida que un campo sea un **CNPJ** o un **CPF** en Brasil.

```
@CPF(message = "El CPF ingresado no es válido")
private String cpf;
```

12. '@Valid'

- Se usa para validar objetos anidados. Cuando una entidad tiene otra entidad dentro, puedes usar '@Valid' para asegurarte de que las validaciones en los campos anidados también se ejecuten.

```
@Valid
private Direccion direccion;
```

13. '@NotEmpty' y '@NotBlank'

- '@NotEmpty': Se asegura de que la cadena, lista o colección no esté vacía.
- '@NotBlank': Valida que una cadena no esté vacía y que no sea solo espacios en blanco.

```
@NotEmpty(message = "El nombre no puede estar vacío")
private String nombre;

@NotBlank(message =
    "El campo de dirección no puede estar vacío o tener solo espacios en blanco")
private String direccion;
```

14. '@Range' (de Hibernate Validator)

- Valida que un número esté dentro de un rango definido.

```
@Range(min = 1, max = 10, message =
    "La calificación debe estar entre 1 y 10")
private int calificacion;
```

15. '@NotNull' ()

- Verifica que el valor de un campo no sea null. Ideal para campos que siempre deben tener un valor.
- Uso: Se aplica a cualquier tipo de datos.

```
@NotNull(message = "El campo no puede ser nulo")
private String nombre;
```

Conclusión Además de '@NotNull', tienes muchas otras anotaciones para validar diversos tipos de datos en tus entidades. Estas validaciones se ejecutan automáticamente cuando envías datos a tu API, y si los datos no cumplen con las restricciones, puedes devolver un mensaje de error claro al usuario.

Puedes combinar múltiples anotaciones en los mismos campos para aplicar varias validaciones.

Usar validaciones como estas ayuda a garantizar la integridad de los datos en tu aplicación y evita que lleguen datos incorrectos o incompletos a la base de datos.